

02285 AI and MAS, F26
Exercises for week 4, 24/2-26
SOLUTIONS

When referring to the **POP algorithm** below we refer to the algorithm provided on today's slides and described in detail in Section 11.3 of Russell & Norvig **2nd edition** (relevant section available on DTU Learn).

Exercise 1 (Partially ordered plans)

Define the action schemas for the problem of putting on shoes and socks and a hat and coat, assuming that there are no preconditions for putting on the hat and coat. Give a partial-order plan that is a solution, and show that there are 180 different linearisations of this solution. You don't necessarily have to apply the POP algorithm to find the partial-order plan in this exercise, you just have to find the plan and show it as a directed graph (like for the socks-and-shoes problems on today's slides).

SOLUTION.

ACTION: *LeftSock*

PRECONDITION:

EFFECT: *LeftSockOn*

ACTION: *LeftShoe*

PRECONDITION: *LeftSockOn*

EFFECT: *LeftShoeOn*

ACTION: *RightSock*

PRECONDITION:

EFFECT: *RightSockOn*

ACTION: *RightShoe*

PRECONDITION: *RightSockOn*

EFFECT: *RightShoeOn*

ACTION: *Hat*

PRECONDITION:

EFFECT: *HatOn*

ACTION: *Coat*

PRECONDITION:

EFFECT: *CoatOn*

The graph is induced by the following partial order (has the following paths from start to finish):

$Start \prec HatOn \prec Finish$
 $Start \prec CoatOn \prec Finish$
 $Start \prec LeftSock \prec LeftShoe \prec Finish$
 $Start \prec RightSock \prec RightShoe \prec Finish$

There are 6 actions in this partial-order plan. They have $6! = 720$ permutations. However, permutations where *LeftShoe* comes before *LeftSock* are not linearisations of the partial-order plan, and similarly for *RighShoe* and *RightSock*. So in total we get $\frac{6!}{4} = 180$ linearisations.

Exercise 2 (POP Algorithm)

Consider again the milk-and-bananas domain with the following action schemas:

ACTION: *Buy*(x, y) (buy item x at location y)
 PRECONDITION: $Buyable(x) \wedge Place(y) \wedge At(y) \wedge Sells(y, x)$
 EFFECT: $Have(x)$

ACTION: *Go*(x, y)
 PRECONDITION: $Place(x) \wedge Place(y) \wedge At(x)$
 EFFECT: $At(y) \wedge \neg At(x)$

Assume we are given the following initial state:

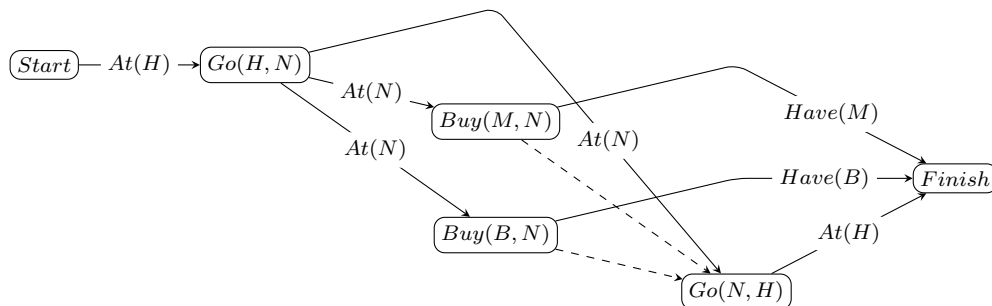
$Buyable(Milk) \wedge Buyable(Bananas) \wedge Buyable(Drill) \wedge Place(Home) \wedge Place(Netto) \wedge Place(ToolShop) \wedge At(Home) \wedge Sells(Netto, Milk) \wedge Sells(Netto, Bananas) \wedge Sells(ToolShop, Drill)$.

- a) What are the rigid predicates in this problem (cf. the exercise session from last week)?

SOLUTION. *Buyable*, *Place* and *Sells*.

- b) Use the POP algorithm to compute a partial-order plan for the following goal: $At(Home) \wedge Have(Milk) \wedge Have(Bananas)$. Whenever you encounter an open precondition using a rigid predicate, just make sure the precondition holds in the initial state, without drawing an edge (to keep the graph clutter-free). If such an open precondition does *not* hold in the initial state, you are enforced to backtrack.

SOLUTION. The result is the graph shown on today's slides (POP example: milk-and-bananas) as well as here:



Note that preconditions and effects are not shown in the figure, making it harder to see when there is a conflict. In general, it is advisable to add preconditions and effects to the graphs as in the examples given in Russell & Norvig.

- c) How many linearisations does your solution have? Are those linearisations optimal solutions to the problem? Will the POP algorithm necessarily find an optimal solution to this particular planning problem?

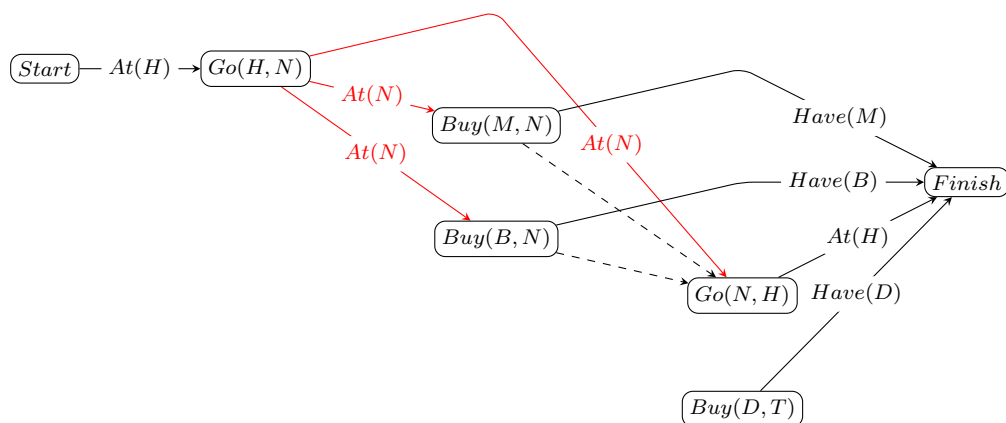
SOLUTION. It is seen from the graph on the slide that there is only a choice between the order of $Buy(Bananas)$ and $Buy(Milk)$. Hence only 2 linearisations (in this particular application of the POP algorithm). These solutions are indeed optimal. But they didn't necessarily have to be. Consider removing $Go(N, H)$ from the graph on the slide. Then there is an open precondition $At(H)$ of $Finish$. We could instead choose to achieve that by the action $Go(ToolShop, Home)$. In that case, we would have a new open precondition $At(ToolShop)$ to achieve. It could be achieved by $Go(Netto, Toolshop)$, producing a new open precondition $At(Netto)$. This latter open precondition could be achieved by the existing actions $Buy(B)$ and $Buy(M)$. But the resulting partially ordered plan is not optimal: after buying the stuff at Netto, the agent passes by the ToolShop on its way home.

- d) Now use the POP algorithm to compute a partial-order plan for the following goal: $At(Home) \wedge Have(Milk) \wedge Have(Bananas) \wedge Have(Drill)$. Note that you can solve this problem by continuing to run the POP algorithm from the graph produced in the previous question with one additional open precondition $Have(Drill)$ of the action $Finish$. You might be forced to backtrack.

SOLUTION. We need to achieve the new open precondition $Have(Drill)$. This can only be achieved by $Buy(Drill, Toolshop)$, so we add that action to the graph and add ordering constraint (edge)

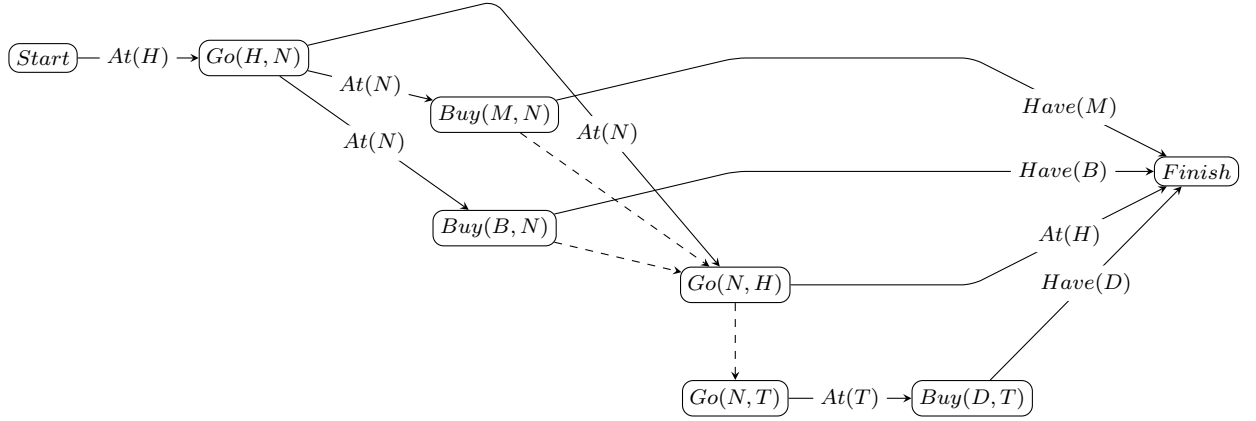
$$Buy(Drill, ToolShop) \overset{Have(Drill)}{\prec} Finish.$$

This doesn't introduce any new conflicts. The graph now looks like this:

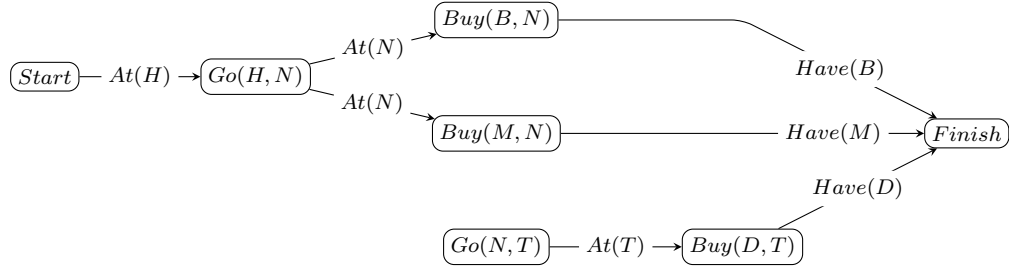


Now the new action $Buy(D, T)$ has a new open precondition $At(T)$. We achieve this by either the action $Go(N, T)$ or the action $Go(H, T)$. If we choose $Go(N, T)$, then it introduces a conflict with the edges labelled $At(N)$ (the red ones), as $Go(N, T)$ has the negative effect $\neg At(N)$. So

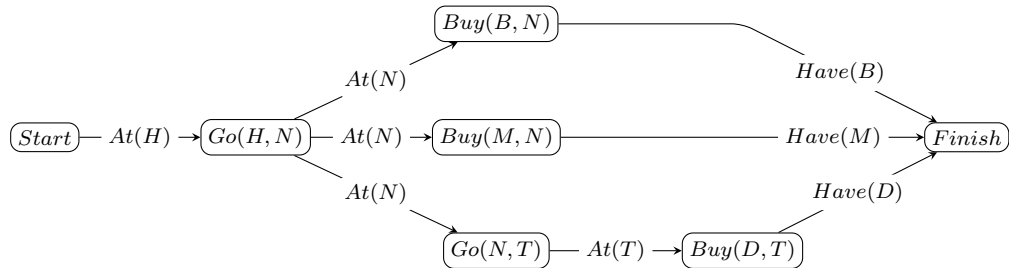
we would have to put $Go(N, T)$ before $Go(H, N)$ or after $Go(N, H)$. Let us suppose we try to put it after $Go(N, H)$:



Now $Go(N, T)$ has the open precondition $At(N)$. There is no way we can achieve $At(N)$ without conflicting with the negative effect $\neg At(N)$ of $Go(N, H)$, so we need to backtrack and try something else. It can be shown that putting $Go(N, T)$ before $Go(H, N)$ doesn't help either. In fact, choosing $Go(H, T)$ instead of $Go(N, T)$ in the first place will also lead to unsolvable conflicts. So we're stuck. We simply have to backtrack and get rid of some of the original graph. An important point is that it doesn't matter what step of the original graph building we omit. The obvious choice is to get rid of $Go(N, H)$. Then the graph looks like this:

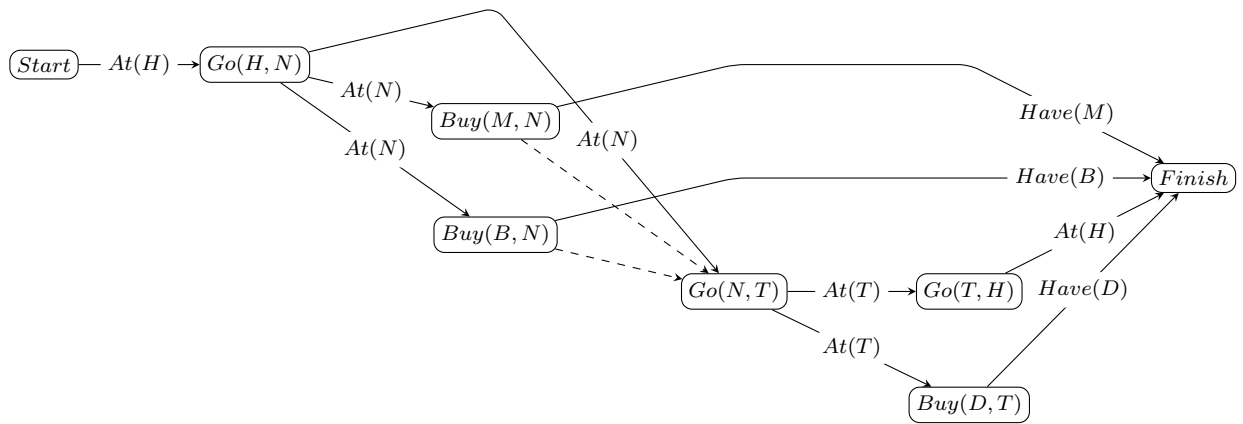


Now we have two open preconditions: $At(N)$ of $Go(N, T)$ and $At(H)$ of $Finish$. Let us deal with $At(N)$ first:



As $Go(N, T)$ has negative effect $\neg At(N)$, it needs to come after $Buy(B, N)$ and $Buy(M, N)$.

Adding the corresponding edges and the final $Go(T, H)$ action, we finally get:



The only thing we have to add now is a ordering constraint putting $Buy(D, T)$ before $Go(T, H)$, and we're done.

Exercise 3 (Delete-Relaxation and POP)

Consider a *delete-relaxed* planning problem P^+ , that is, where all negative effects are removed. This exercise concerns whether the relaxed problem P^+ is easier to solve using POP than the original problem P .

- a) Show that when running the POP algorithm on P^+ a conflict can never arise.

SOLUTION. A conflict can only arise when an action D has an effect e that is inconsistent with the literal c appearing in a labelled ordering constraint $A \prec^c B$. But all literals c appearing in labelled ordering constraints are effects of actions, and so is e , so they must both be positive. But then they can't be inconsistent.

- b) Consider a POP algorithm working in a best-first manner, where a heuristic chooses the next open precondition to be resolved. Can such an algorithm ever be forced to backtrack on a delete-relaxed problem P^+ ?

SOLUTION. Yes. Consider a version of the milk-and-bananas problem, where the agent can buy Milk on Mars, but has no action of going to Mars. Then we can still choose $Buy(Milk, Mars)$ (buy milk at Mars) to achieve the open precondition $Have(Milk)$, but afterwards we will not be able to satisfy the open precondition $At(Mars)$. Hence, we will have to backtrack and consider buying the milk somewhere less exotic, e.g. at Netto.

Exercise 4 (HTN planning)

Solve Exercise 11.2 of R&N: "You have a number of trucks with which to deliver a set of packages. Each package starts at some location on a grid map, and has a destination somewhere else. Each truck is directly controlled by moving forward or turning. Construct a hierarchy of high-level actions for this problem (you are free to decide on the details of the problem). What knowledge about the solutions does your hierarchy encode?"

Additional question: Solve a simple instance of the problem by refinements of you high-level actions.

SOLUTION. We use constants T_i for trucks, constants P_i for packages and constants L_i for locations. The initial state is:

$$\begin{aligned} & \bigwedge \{Truck(T_i) \mid T_i \text{ is a truck}\} \wedge \bigwedge \{Package(P_i) \mid P_i \text{ is a package}\} \wedge \\ & \bigwedge \{Location(L_i) \mid L_i \text{ is a location}\} \wedge \\ & \bigwedge \{Neighbour(L_i, L_j, dir) \mid L_j \text{ is neighbour of } L_i \text{ in direction } dir \in \{S, W, N, E\}\} \wedge \\ & DirRight(N, E) \wedge DirRight(E, S) \wedge DirRight(S, W) \wedge DirRight(W, N) \wedge \\ & \bigwedge \{At(\alpha, \beta) \mid \alpha \text{ is a truck or package and } \beta \text{ is its location}\} \wedge \\ & \bigwedge \{Dir(T_i, dir) \mid \text{the truck } T_i \text{ is pointing direction } dir \in \{S, W, N, E\}\} \wedge \\ & \bigwedge \{Dest(P_i, L_j) \mid \text{package } P_i \text{ has destination } L_j\}. \end{aligned}$$

We build the hierarchy of actions top down, starting with the top level action *Act*.

ACTION : *Act*

PRECOND :

EFFECT : $\bigwedge \{At(P_i, L_j) \mid P_i \text{ has destination } L_j\}$

REFINEMENT : *Act*

PRECOND : $Truck(truck) \wedge Package(pack) \wedge At(truck, start) \wedge At(pack, ploc) \wedge Dest(pack, end) \wedge ploc \neq end$

STEPS : $[NavToPack(truck, pack, start, ploc), NavPackToDest(truck, pack, ploc, dest), Act]$

REFINEMENT : *Act*

PRECOND : $\bigwedge \{At(P_i, L_j) \mid P_i \text{ has destination } L_j\}$

STEPS : $[\]$

Note that the action schemes and refinements above make use of the goal statement $\bigwedge \{At(P_i, L_j) \mid P_i \text{ has destination } L_j\}$. This makes these actions dependent on the initial state, which would be illegal in PDDL (the domain has to be independent of initial state and goal), but is acceptable in HTN.

The next level down in the hierarchy consists of the high-level actions *NavToPack* and *NavPackToDest*:

ACTION : *NavToPack*(*truck*, *pack*, *start*, *end*)

PRECOND : $Truck(truck) \wedge Package(pack) \wedge At(truck, start) \wedge At(pack, end) \wedge start \neq end$

EFFECT : $At(truck, end) \wedge \neg At(truck, start)$

REFINEMENT : *NavToPack*(*truck*, *pack*, *start*, *end*)

PRECOND : $Truck(truck) \wedge Package(pack) \wedge At(truck, start) \wedge At(pack, end) \wedge start \neq end \wedge Dir(truck, dir)$

STEPS : $[NavTruck(truck, dir, start, end)]$

ACTION : $NavPackToDest(truck, pack, start, end)$
 PRECOND : $Truck(truck) \wedge Package(pack) \wedge At(truck, start) \wedge At(pack, start) \wedge Dest(pack, end) \wedge start \neq end$
 EFFECT : $At(truck, end) \wedge At(pack, end) \wedge \neg At(truck, start) \wedge \neg At(pack, start)$

 REFINEMENT : $NavPackToDest(truck, pack, start, end)$
 PRECOND : $Truck(truck) \wedge Package(pack) \wedge At(truck, start) \wedge At(pack, start) \wedge Dest(pack, end) \wedge start \neq end \wedge Dir(truck, dir)$
 STEPS : $[NavTruckWPack(truck, pack, dir, start, end)]$

The next level down in the hierarchy consists of the high-level actions *NavTruck* and *NavTruckWPack*:

ACTION : $NavTruck(truck, dir, start, end)$
 PRECOND : $Truck(truck) \wedge At(truck, start) \wedge Dir(truck, dir) \wedge Location(end)$
 EFFECT : $At(truck, end) \wedge \neg At(truck, start)$

 REFINEMENT : $NavTruck(truck, dir, start, end)$
 PRECOND : $Truck(truck) \wedge At(truck, start) \wedge Dir(truck, dir) \wedge Location(end) \wedge start \neq end$
 STEPS : $[MoveForward(truck, dir, start, next), NavTruck(truck, dir, next, end)]$

 REFINEMENT : $NavTruck(truck, dir, start, end)$
 PRECOND : $Truck(truck) \wedge At(truck, start) \wedge Dir(truck, dir) \wedge Location(end) \wedge start \neq end$
 STEPS : $[TurnRight(truck, dir, start, newdir), NavTruck(truck, newdir, start, end), TurnLeft(truck, newdir, end, dir)]$

 REFINEMENT : $NavTruck(truck, dir, start, end)$
 PRECOND : $Truck(truck) \wedge At(truck, start) \wedge Dir(truck, dir) \wedge Location(end) \wedge start = end$
 STEPS : $[\]$

We have left out a refinement where the truck starts out by turning left as a left turn can always be implemented as a sequence of right turns. This is simply to make the specification shorter. We could of course also have specified a general turn action, where the turn direction was given as an additional parameter. The high-level action *NavTruckWPack* is specified similarly as *NavTruck* above, except we need the extra package parameter, and it refers to *MoveForwardWPack* instead of *MoveForward*. At the lowest level we have the primitive actions *MoveForward*, *TurnRight* and

MoveForwardWPack:

ACTION : *MoveForward*(truck, dir, start, end)

PRECOND : *Truck*(truck) $\wedge$ *At*(truck, start) $\wedge$ *Dir*(truck, dir) $\wedge$ *Neighbour*(start, end, dir)

EFFECT : *At*(truck, end) $\wedge$ $\neg$ *At*(truck, start)

ACTION : *TurnRight*(truck, dir, start, newdir)

PRECOND : *Truck*(truck) $\wedge$ *At*(truck, start) $\wedge$ *Dir*(truck, dir) $\wedge$ *DirRight*(dir, newdir)

EFFECT : *Dir*(truck, newdir) $\wedge$ $\neg$ *Dir*(truck, dir)

ACTION : *MoveForwardWPack*(truck, pack, dir, start, end)

PRECOND : *Truck*(truck) $\wedge$ *Package*(pack) $\wedge$ *At*(truck, start) $\wedge$ *At*(pack, start) $\wedge$
Dir(truck, dir) $\wedge$ *Neighbour*(start, end, dir)

EFFECT : *At*(truck, end) $\wedge$ *At*(pack, end) $\wedge$ $\neg$ *At*(truck, start) $\wedge$ $\neg$ *At*(pack, start)

The knowledge about solutions encoded in the hierarchy (plus a few additional comments about the hierarchy):

- a) *Act* encodes the knowledge that: 1) any solution can be composed of moving one package at a time to its destination; 2) moving a package to its destination can be done by first moving a truck to the package, and then use the truck to move the package to its destination.
- b) It is not necessary to worry about the direction of the truck at the highest levels. Direction is only taken into account at the third level of the hierarchy.
- c) The truck always returns to its original orientation after having navigated to a package or navigated a package to a goal (this is exactly to be able to not worry about orientation at the highest levels).
- d) Note that we do not have explicit pickup and drop actions. These are encoded implicitly by separating between actions where a truck is moving without a package and where it is.
- e) A truck can only carry one package at a time.

It can be proved that the action hierarchy above has the downward refinement property.

Exercise 5 (HTN and attached procedures in Sokoban)

Optional: In Sokoban, there are essentially two types of actions: moves and pushes. Concerning moves, it seems advantageous to introduce a high-level action for “jumping” from one cell to another reachable cell. However, such a high-level action would need a precondition for checking whether the to-cell is reachable from the from-cell by a sequence of moves (no pushes). We can not construct literals for recording such information in pure classical planning. Why not?

A possible solution is to introduce so-called *attached procedures*. These are also sometimes called *code-call conditions*. The idea is to allow actions to have preconditions that need to be checked by external computations. An example would be a precondition *Reachable*(x, y) that checks whether the cell y is reachable from cell x . This can easily be computed from the state description by a procedure external to the planner.

Construct a hierarchy of high-level actions for the Sokoban domain, possibly using attached procedures. How does your high-level actions reduce the search needed in solving Sokoban problems? What knowledge about solving Sokoban puzzles does your hierarchy encode?